UNIVERSITE CHOUAIB DOUKKALI
ECOLE NATIONALE DES SCIENCES APPLIQUEES D'EL JADIDA

جامعة شعيب الدكالي
Université Chouaib Doukkali

Cybersécurité & Confiance Numérique (CCN)

Algorithmique et programmation avancée avec Python

Pr. Fatima zahra SALMAM
salmam.f@ucd.ac.ma

python™

1

Partie IV :

- ✓ Les exceptions
- ✓ Les expressions régulières

2

Les exceptions python

Les langages de programmation comme Python contiennent un système de gestion des exceptions.

- **Qu'est-ce qu'une exception ?**
  - Une exception est définie comme une anomalie de l'exécution d'un programme requérant une action spéciale, en général l'arrêt de l'exécution.
  - Le plus souvent une exception correspond à une erreur que Python rencontre lorsqu'il tente d'exécuter les lignes de code qu'on lui soumet.
  - Par exemple, un problème de syntaxe, une variable ou objet qui prend une valeur aberrante (par exemple diviser par 0, parcourir une liste au-delà du nombre d'éléments, etc.).
- **Exemple:**

```
>>> 5/0
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ZeroDivisionError: integer division or modulo by zero
```


Les exceptions python

- Le système de gestion des exceptions évite que votre programme « plante » en prévoyant vous-même les sources d'erreurs éventuelles.
- Voici un exemple dans lequel on demande à l'utilisateur d'entrer un nombre entier, puis on affiche ce nombre.

```
>>> nb = int(input("Entrez un nombre entier : "))
Entrez un nombre entier : 23
>>> print(nb)
23
```

- La fonction **input()** demande à l'utilisateur de saisir une chaîne de caractères. Cette chaîne de caractères est ensuite transformée en nombre entier avec la fonction **int().**
- Si l'utilisateur ne rentre pas un nombre, voici ce qui se passe :

```
>>> nb = int(input("Entrez un nombre entier : "))
Entrez un nombre entier : ATCG
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: invalid literal for int() with base 10: 'ATCG'
```

Les exceptions python

- L'erreur provient de la fonction **int()** qui n'a pas pu convertir la chaîne de caractères "ATCG" en nombre entier, ce qui est parfaitement normal. En termes plus techniques, on dira que « Python a levé une exception de type ValueError ».
- Il y a de nombreux types d'exceptions différents! Le nom de l'exception apparaît toujours comme le premier mot de la dernière ligne du message d'erreur.
- Si nous lancions ces lignes de code sous forme de script (du style python script.py), cet exemple conduirait à l'arrêt de l'exécution du programme.
- Le jeu d'instructions **try / except** permet de tester l'exécution d'une commande et d'intervenir en cas de levée d'exception.

```
>>> try:
...     nb = int(input("Entrez un nombre entier : "))
... except:
...     print("Vous n'avez pas entré un nombre entier !")
...
Entrez un nombre entier : ATCG
Vous n'avez pas entré un nombre entier !
```


5

Les exceptions python

- Dans l'exemple précèdent, l'exception levée par la fonction **int()** (qui ne peut pas convertir "ATCG" en nombre entier) est interceptée et déclenche l'affichage du message d'avertissement.
- On peut ainsi redemander sans cesse un nombre entier à l'utilisateur, jusqu'à ce que celui-ci en rentre bien un.

```
>>> while True:
...     try:
...             nb = int(input("Entrez un nombre entier : "))
...             print("Le nombre est", nb)
...             break
...     except:
...             print("Vous n'avez pas entré un nombre entier !")
...             print("Essayez encore")
...
Entrez un nombre entier : ATCG
Vous n'avez pas entré un nombre entier !
Essayez encore
Entrez un nombre entier : toto
Vous n'avez pas entré un nombre entier !
Essayez encore
Entrez un nombre entier : 3.2
Vous n'avez pas entré un nombre entier !
Essayez encore
Entrez un nombre entier : 55
Le nombre est 55
```

- Notez que dans cet exemple, l'instruction **while True** est une boucle infinie car la condition **True** est **toujours vérifiée**.
- L'arrêt de cette boucle est alors forcé par la commande **break** lorsque l'utilisateur a effectivement entré un nombre entier.

6

Les exceptions python

- La gestion des exceptions est très utile dès lors que des données extérieures entrent dans un programme Python, que ce soit directement par l'utilisateur (avec la fonction input()) ou par des fichiers. Cela est fondamental si vous distribuez votre code à la communauté : si les utilisateurs ne connaissent pas Python, **un message comme** Vous n'avez pas entré un nombre entier! **reste plus clair que** ValueError: invalid literal for int() with base 10: 'ATCG'

- Vous pouvez par exemple vérifier qu'un fichier a bien été ouvert.

```
>>> nom = "toto.pdb"
>>> try:
...     with open(nom, "r") as fichier:
...         for ligne in fichier:
...             print(ligne)
... except:
...     print("Impossible d'ouvrir le fichier", nom)
```

- Si une erreur est déclenchée, c'est sans doute que le fichier n'existe pas à l'emplacement indiqué sur le disque ou que vous n'avez pas les droits pour le lire.

7

Les exceptions python

Cibler les erreurs:

- Il est également possible de spécifier le type d'erreur à gérer. Le premier exemple que nous avons étudié peut s'écrire :

```
>>> try:
...     nb = int(input("Entrez un nombre entier : "))
... except ValueError:
...     print("Vous n'avez pas entré un nombre entier !")
...
Entrez un nombre entier : ATCG
Vous n'avez pas entré un nombre entier !
```

- Ici, on intercepte une exception de type **ValueError**, ce qui correspond bien à un problème de **conversion avec int().**

- Attention, si vous précisez le type d'exception comme **ValueError**, le except ValueError **n'empêchera pas la levée d'une autre exception**.

8

Les exceptions python

```
>>> try:
...     nb = int(variable)
... except ValueError:
...     print("Vous n'avez pas entré un nombre entier !")
...
Traceback (most recent call last):
  File "<stdin>", line 2, in <module>
NameError: name 'variable' is not defined
```

- Ici l'exception levée est de type **NameError** car variable n'existe pas. Alors que si vous mettez **except tout court**, **cela intercepte n'importe quelle exception.**

```
>>> try:
...     nb = int(variable)
... except:
...     print("Vous n'avez pas entré un nombre entier !")
...
Vous n'avez pas entré un nombre entier !
>>>
```

- Vous voyez qu'ici cela pose un nouveau problème : le message d'erreur ne correspond pas à l'exception levée !

9

Les exceptions python

```
>>> try:
...     nb = int(variable)
... except ValueError:
...     print("Vous n'avez pas entré un nombre entier !")
...
Traceback (most recent call last):
  File "<stdin>", line 2, in <module>
NameError: name 'variable' is not defined
```

- Ici l'exception levée est de type **NameError** car variable n'existe pas. Alors que si vous mettez **except tout court**, **cela intercepte n'importe quelle exception.**

```
>>> try:
...     nb = int(variable)
... except:
...     print("Vous n'avez pas entré un nombre entier !")
...
Vous n'avez pas entré un nombre entier !
>>>
```

- Vous voyez qu'ici cela pose un nouveau problème : le message d'erreur ne correspond pas à l'exception levée !

10

Les exceptions python

- Il est possible en python d'exécuter des instructions en fonction du type d'erreur. Par exemple si la valeur est 0 on aimerait afficher un message et si le dénominateur est du texte on aimerait pouvoir afficher un autre message à l'utilisateur.

```
>>> v = 0
>>> w = 5
>>> try:
... w/v
... print("Ok pas erreur")
... except TypeError:
... print("Merci d'utiliser des chiffres")
... except ZeroDivisionError:
... print("Merci de ne pas utiliser le 0")
...
Merci de ne pas utiliser le 0
```


11

Les exceptions python

- et dans le cas où la variable v vaut " olivier ":

```
>>> v = "olivier"
>>> w = 5
>>> try:
... w/v
... print("Ok pas erreur")
... except TypeError:
... print("Merci d'utiliser des chiffres")
... except ZeroDivisionError:
... print("Merci de ne pas utiliser le 0")
...
Merci d'utiliser des chiffres
```

- On utilise le mot clé finally pour exécuter des instructions quel que soit les erreurs générées ou non (et même si il y a présence d'un return). Dans tous les cas les instructions placées dans finally seront executées.

```
try:
     pass
except:
     pass
finally:
     print("Execution")
```


12

Les exceptions python

Remarques:

- Nous vous conseillons vivement de toujours préciser le type d'exception dans vos except.
- Cela évite d'intercepter une exception que vous n'aviez pas prévue.
- Il est possible d'intercepter plusieurs types d'exceptions en passant un tuple à except, par exemple : except (Exception1, Exception2).
- Par ailleurs, ne mettez pas trop de lignes dans le bloc du try. Dans un tel cas, il peut être très pénible de trouver la ligne qui a conduit à l'exécution du except.
- Pire encore, il se peut que des lignes que vous aviez prévues ne soient pas exécutées !
- Donc gardez des choses simples dans un premier temps, comme par exemple tester les conversions de type ou vérifier qu'un fichier existe bien et que vous pouvez l'ouvrir.

- Il existe de nombreux types d'exception comme RuntimeError, TypeError, NameError, IOError, etc. Vous pouvez aller voir la liste complète sur le site de Python.

Les exceptions python: Exercice

- Ecrivez un programme qui demande à l'utilisateur de saisir un nombre et affiche son inverse. Gérer toutes les exceptions possibles.

Partie IV :

- ✓ Les exceptions
- ✓ Les expressions régulières

15

Les expressions régulières

- Les expressions régulières (rationnelles) sont utilisées dans quasiment tous les langages. C'est un outil très puissant qui permet de vérifier si le contenu d'une variable a la forme de ce que l'on attend.
- Par exemple si on récupère un numéro de téléphone, on s'attend à ce que la variable soit composée de nombres et d'espaces (ou de tiret) mais rien de plus.
- Les expressions régulières permettent non seulement de vous avertir d'un caractère non désiré mais également de supprimer/modifier tous ceux qui ne sont pas désirables.
- Concrètement, les expressions régulières vont nous permettre de vérifier la présence de certains caractères ou suites de caractères dans une expression afin de valider la forme de données envoyées par l'utilisateur via une fonction **input()** par exemple.

16

Les expressions régulières

Les bases

- On utilise des symboles qui ont une signification:

```
. ^ $ * + ? { } [ ] \ | ( )
```

```
.      Le point correspond à n'importe quel caractère.
^      Indique un commencement de segment mais signifie aussi "contraire de"
$      Fin de segment
[xy]   Une liste de segment possibble. Exemple [abc] équivaut à : a, b ou c
(x|y)  Indique un choix multiple type (ps|ump) équivaut à "ps" OU "UMP"
\d     le segment est composé uniquement de chiffre, ce qui équivaut à [0-9].
\D     le segment n'est pas composé de chiffre, ce qui équivaut à [^0-9].
\s     Un espace, ce qui équivaut à [ \t\n\r\f\v].
\S     Pas d'espace, ce qui équivaut à [^ \t\n\r\f\v].
\w     Présence alphanumérique, ce qui équivaut à [a-zA-Z0-9_].
\W     Pas de présence alphanumérique [^a-zA-Z0-9_].
\      Est un caractère d'échappement
```


17

Les expressions régulières

- Il est possible d'imposer le nombre d'occurrences avec la syntaxe suivante:

A{2} : on attend à ce que la lettre A (en majuscule) se répète 2 fois consécutives.

BA{1,9} : on attend à ce que le segment BA se répète de 1 à 9 fois consécutives.

BRA{,10} : on attend à ce que le segment BRA ne soit pas présent du tout ou présent jusqu'à 10 fois consécutives.

VO{1,} : on attend à ce que le segment VO soit présent au moins une fois.

Symbole	Nb Caractères attendus	Exemple	Cas possibles
?	0 ou 1	GR (.)?S	GRS, GROS, GRIS, GRAS
+	1 ou plus	GR (.)+S	GROS, GRIS, GRAS
*	0, 1 ou plus	GR (.)*S	GRS,GROS ,GRIIIS,GROlivierS

18

Les expressions régulières

- Python nous permet d'exploiter les expressions régulières via son module standard **re**.
- Nous allons donc utiliser des fonctions qui vont permettre d'effectuer des recherches de motifs dans des chaines, des remplacements, etc.
- Une expression régulière en soi n'est qu'un motif. Nous allons donc utiliser des fonctions qui vont permettre d'effectuer des recherches de motifs dans des chaines, des remplacements, etc.
- La fonction **compile()** compile un motif vers une expression rationnelle compilée qu'on va pouvoir utiliser avec les méthodes **match()** et **search()**.
- On utilisera **compile()** lorsqu'on voudra sauvegarder une expression régulière pour la réutiliser dans un programme.

19

Les expressions régulières

- La fonction **match()** prend également un schéma de recherche en premier argument et une chaine dans laquelle effectuer la recherche en deuxième argument et renvoie l'objet de correspondance trouvé si zéro ou plus caractères au début de la chaine correspondent à l'expression régulière ou None si la chaîne ne correspond pas au motif.

```
>>> import re
>>> p = re.compile('[a-z]+')
```

- Maintenant, vous pouvez tester des correspondances de la RE **[a-z]+** avec différentes chaînes.
- Une chaîne vide ne doit pas correspondre, puisque **+** indique « **une ou plusieurs occurrences** ». **match()** doit renvoyer **None** dans ce cas, ce qui fait que l'interpréteur n'affiche rien.

20

Les expressions régulières

Exemple :

```
>>> print(p.match(""))
None
```

Sur une chaîne qui correspond, par exemple **tempo**. Dans ce cas, **match()** renvoie un objet correspondance, vous pouvez ainsi stocker le résultat dans une variable pour une utilisation ultérieure.

```
>>> m = p.match('tempo')
>>> m
<re.Match object; span=(0, 5), match='tempo'>
```


21

Les expressions régulières

Vous pouvez interroger l'objet correspondance pour obtenir des informations sur la chaîne qui correspond.

Les instances d'objets correspondances possèdent plusieurs méthodes et attributs ; les plus importants sont :

Méthode/Attribut	Objectif
group()	Renvoie la chaîne de caractères correspondant à la RE
start()	Renvoie la position de début de la correspondance
end()	Renvoie la position de fin de la correspondance
span()	Renvoie un *n*-uplet contenant les positions (début, fin) de la correspondance

22

Les expressions régulières

Exemple:

```
>>> m.group()
'tempo’
>>> m.start(), m.end()
(0, 5)
>>> m.span()
(0, 5)
```


23

Les expressions régulières

- La fonction **search()** permet d’analyser une chaine passée en deuxième argument à la recherche du premier emplacement où l’expression régulière passé en premier argument trouve une correspondance, et renvoie l’objet de correspondance trouvé ou None si aucune position dans la chaîne ne valide le motif.
- En d'autres termes, la méthode **search()** d'un motif analyse toute la chaîne, afin de trouver une correspondance potentielle qui ne commence pas à zéro.

```
>>> print(p.match('::: message’))
None
>>> m = p.search('::: message’);
print(m) <re.Match object; span=(4, 11), match='message’>
>>> m.group()
'message’
>>> m.span()
(4, 11)
```


24

Les expressions régulières

- Dans les programmes réels, la façon de faire la plus courante consiste à stocker l'objet correspondance dans une variable, puis à vérifier s'il vaut None. Généralement, cela ressemble à ceci :

```
p = re.compile( ... )
m = p.match( 'string goes here' )
        if m:
                print('Match found: ', m.group())
        else:
                print('No match')
```


25

Les expressions régulières

- Il existe deux méthodes de motifs qui renvoient toutes les correspondances pour un motif. La méthode **findall()** et la métode **finditer()**
- La fonction **findall()** renvoie toutes les correspondances de l'expression régulière passée en argument sous forme d'une liste de chaînes.
- La chaîne est analysée de la gauche vers la droite, et les correspondances sont renvoyées dans l'ordre où elles sont trouvées.

Exemple :

```
>>> import re
>>> p = re.compile(r'\d+')
>>> p.findall("Bonjour 11 Aurevoir 22 Hello 333 By 44")
 ['11', '22', '33', '44']
```

Remarque: Le préfixe **r**, qui indique une chaîne brute littérale, est nécessaire dans cet exemple pour reconnaître les séquences d'échappement!

26

Les expressions régulières

- La méthode **finditer()** renvoie une séquence d'instances d'objets correspondances en tant qu'itérateur :

```
iterator = p.finditer('Bonjour 11 Aurevoir 22 Hello 33 By 44 ')
>>> iterator
<callable_iterator object at 0x...>
>>> for match in iterator:
        ... print(match.span())
        ...
(8, 10)
(20, 22)
(29, 31)
(35, 37)
```


27

Les expressions régulières

```
>>> import re
>>> masque1 = "ier" #correspond à ‘ier’
>>> masque2 = "^[0-9]{10}$" #correspond à une chaine de 10 chiffres exactement

>>> r1 = re.compile(masque1)
>>> r2 = re.compile(masque2)

>>> c1 = " Je m’apelle Pierre, hier j’étais … "
>>> c2 = "0636765787"
>>> c3 = "1234567"

>>> print(re.search(r1,c1))
<re.Match object; span=(14,17), match=‘ier’>

>>> print(re.match(r1,c1))
None

>>> print(re.findall(r1,c1))
[‘ier’, ‘ier’]

>>>print(re.search(r2, c2))
<re.Match object; span=(0,10), match=‘0636765787’>

>>>print(re.search(r2, c3))
None
```

28

Les expressions régulières: Exercice

Ecrivez un programme qui permet de tester si un e-mail est valide ou non.

Le programme prend une liste des e-mails et affiche les e-mails valides et non valides.

Si on considère l'exemple suivant:

```
M =["salmam@mail.com", "salmam@mail.ca", "sal_8@mail.com", "@mail.com", "salmam@mailcom"]
```

Le programme doit afficher:

```
Ce mail : salmam@mail.com est valide
Erreur ce mail : salmam@mail.ca est non valide
Ce mail : sal_8@mail.com est valide
Erreur ce mail : @mail.com est non valide
Erreur ce mail : salmam@mailcom est non valide
```